DATA MANAGEMENT CARPENTRY · EPISODE 2

Choosing the Right File Structures

Diagnosing structural issues, core design principles, and practical techniques to reorganise inherited research folders.

EPISODE SCOPE

Questions & Objectives

Core Questions

Why does folder organisation matter?

What makes a good folder structure?

How to spot problems in file structures?

How to move, rename, and delete folders?

Learning Objectives

Identify issues with names and organisation.

Move & rename folders with a file browser.

Delete unnecessary or duplicate folders.

Design structures supporting reproducibility.

CURRENT STATE

Inside The Inherited Dataset

LEGACY DIRECTORY SNAPSHOT

```
legacy_dataset/
├── miscellaneous/ (cat_pic.jpg, Untitled.csv)
├── New Folder/ (Image One.png, New Folder (2)/)
├── penguins/ (palmer_penguins.csv, penguin958.jpg)
├── project_data_2025/
│   ├── data_final_v3_LAST_ONE.xlsx
│   ├── Alice_Jones_passport.pdf
│   └── .processed/REAL_ACTUAL_DATA_DO_NOT_DELETE.csv
├── processing_data/
└── RAW_DATA_!!!/
    ├── Data!@#$.csv
    └── raw_images_TEMP/
```

Six Months Later, Could You Answer:

- Where is the authoritative raw data?
- Which files should actually be analysed?
- Which files are temporary or deletable?
- Why are personal passport scans mixed in?

EXERCISE · 5 MINUTES

Challenge: What's Wrong With This Structure?

Working in groups of 2–3:

Explore the inherited dataset for 5 minutes. Identify as many structural problems as you can:

- **Folder names:** Default, generic, or vague?
- **Organisation:** Are related files together?
- **Folder depth:** Excessive nesting or hiding?
- **Duplication:** Redundant scripts or copies?
- **Clarity:** Can a newcomer understand purpose?
- **Consistency:** Are conventions unified?

CHALLENGE SOLUTION & OBSERVATIONS

Common Issues in the Dataset

Default & Vague Names

`New Folder`, `New Folder (2)`, and `miscellaneous` give zero context.

Special Characters

`RAW_DATA_!!!` and `Data!@#$.csv` break scripts and command-line tools.

Hidden Locations

Crucial data tucked in dot-folders (`.processed/`) invisible to default file explorers.

Mixed Content Types

Personal passport PDFs, Python scripts, raw data, and cat photos coexisting in one place.

CORE MOTIVATION

Why Folder Structures Matter

The Project Growth Trap

Projects start with 3 files: `data.csv`, `notes.docx`, `report.docx`.

They rapidly expand into hundreds of scripts, figures, draft manuscripts, manual notes, and raw inputs.

The Golden Rule of Thumb

"A collaborator should be able to understand where files belong without asking you."

Plan for expansion early to avoid breaking internal script paths later.

DESIGN ARCHITECTURE

Principles of Good Folder Structures

Meaningful

Names communicate clear purpose.

Consistent

One naming style across all folders.

Shallow

Max 3–4 levels deep; avoid nesting.

Categorical

Separate raw, code, and figures.

Archivable

Archive obsolete files; don't hoard.

DISCUSSION

Meaningful vs. Consistent Names

Poor Examples

Meaningless:

```
stuff / miscellaneous / new_folder / things
```

Inconsistent Casing & Formats:

```
RawData / processed-data / Analysis Files
```

Good Alternatives

Communicates clear purpose:

```
raw_data / processed_data / analysis / documentation
```

Unified lower_case_snake convention:

```
figures / analysis / data / documentation
```

HIERARCHY DESIGN

Keep Structures Shallow (3–4 Levels)

ANTI-PATTERN: ENCODING METADATA IN PATHS

```
Project/
└── Data/
    └── Raw/
        └── 2025/
            └── 05/
                └── 06/
                    └── Newcastle/
                        └── A_roads/
                            └── UserID12345/
```

Why Deep Nesting Fails:

Click & Typing Fatigue: Requires excessive clicks in GUI and unwieldy path strings in scripts.

Difficult Aggregation: Analyzing all Newcastle data requires looping across hundreds of separate subfolders.

Metadata Risk: If files move, folder-based metadata (dates, cities) is permanently lost. Store metadata **inside files or tabular indices**.

MODULAR ORGANIZATION

Categorical Grouping

Poor: Mixed Junk Drawer

```
project/
├── raw_data.csv
├── final_figure.png
├── manuscript.docx
├── script.py
└── notes.txt
```

Good: Clean Domain Separation

```
project/
├── data/ (raw/ and processed/)
├── analysis/ (code and scripts)
├── figures/ (generated visual plots)
├── documentation/ (protocols and notes)
└── archive/ (obsolete legacy files)
```

PRACTICAL OPERATIONS

How to Move Folders

Click and Drag

Click, hold, drag to parent folder, release.

Best for visible, adjacent folders.

Cut and Paste

Windows/Linux: Right-click > Cut > Paste

macOS: Right-click > Copy > Option+Right-click > Move Item Here

Keyboard Shortcuts

Windows/Linux: `Ctrl + X` then `Ctrl + V`

macOS: `Cmd + X` then `Cmd + V`

EXERCISE & METHODS

Renaming Folders Safely

QUICK CHALLENGE

Rename: `RAW_DATA_!!!`

What is a cleaner, script-friendly alternative?

Target: `raw_data`

How to Rename

- **Right-click** folder > Select **Rename** > Press Enter
- **Windows shortcut:** Select folder > Press F2
- **macOS shortcut:** Select folder > Click name or press `Return`

Caution: Renaming may break script paths and references in documentation.

HYGIENE & GOVERNANCE

Deleting Unnecessary Folders

Deletion Mechanics

- **Drag to Bin:** Desktop (Windows) or Dock (macOS).
- **Shortcuts:** `Delete` (Win) or `Cmd + Delete` (Mac).
- Files rest in the bin and can be restored before purging.



Compliance & Sync Cautions

Local deletion may **not** remove files from cloud syncs (e.g. OneDrive) or institutional automated backups.

For active research, moving obsolete data to `archive/` is far safer than immediate deletion.

DESIGN CHALLENGE

Design Your Own Project Structure

For one of your own research projects:

1. Consider useful folders and how to divide up your files.
2. Draw out **three possible folder hierarchies** on paper.
3. Discuss the pros and cons of each approach with your neighbours.

Goal: Ensure every folder has one unambiguous, clear purpose.

DESIGN REFERENCE

A Flexible, Modular Reference Structure

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── analysis/
│   └── scripts/
├── documentation/
├── outputs/
│   └── figures/
└── archive/
```

Modular Additions:

- `code/` or `src/` for software pipelines.
- `figures/` placed inside analysis or outputs.
- `temp/` for scratch work you know will not be preserved.

There is no single correct answer — consistency and purpose are what count.

CLASS DISCUSSION

What Else Would You Change?

Looking at the inherited dataset with your neighbour:

- Which folders would you **rename**?
- Which folders would you **move**?

- Which folders would you **remove**?
- What **new folders** would you create?

PRACTICAL EXERCISE

Challenge: Repair the Project Structure

Individually or in pairs, perform the repair:

1. Rename unclear folders (eliminate special characters and spaces).
2. Move scattered folders into logical parent locations.
3. Remove unnecessary empty folders.
4. Create distinct locations for: **Data**, **Documentation**, **Analysis**, and **Outputs**.

Focus on making the project clean and understandable — perfection is not required.

EXERCISE SOLUTION

A Reasonable Repaired Outcome

Completed Remediation

- **Renamed:** `RAW_DATA_!!!` → `raw_data`
- **Replaced:** `miscellaneous` decomposed into clear folders
- **Pruned:** Deleted `New Folder (2)` and empty artifacts
- **Categorised:** Dedicated `documentation/` and `analysis/`
- **Flattened:** Reduced nesting depth throughout

Target Result

A newcomer can now find raw inputs, scripts, and documentation within **3 seconds**.

Private passport scans and cat photos have been separated or securely quarantined.

SUMMARY

Episode 2 Key Points

Purpose-Driven: Folder structures exist to help people find and understand data.

Meaningful & Consistent: Use clear names and stick strictly to one naming style.

Domain Separation: Separate data, code, outputs, and documentation into distinct folders.

Avoid Nesting: Keep hierarchy shallow (3–4 levels); never encode metadata in paths.

The Invariant: A good folder structure communicates clearly to collaborators and your future self.

LOOKING AHEAD

Next: Episode 3

We have improved the project's folder structure, but many files still have confusing, arbitrary names. Next, we explore systematic file naming conventions.

File Naming Conventions